Assignment - 1

1) How Java supports platform independency ? What is the role of JVM in it ?

↳ Java follows the principle 'Write Once, Run Anywhere (WORA)'. A Java program is compiled into bytecode, not machine-specific code. This bytecode can run on any system that has a JVM.

The bytecode is platform independent, meaning it can run on any operating system such as Windows, Linux, or macOS, provided that a Java Virtual Machine (JVM) is installed on that system.

→ Role of JVM: The Java Virtual Machine (JVM) acts as a bridge between the bytecode and the underlying hardware. Its main functions include:

- Loading the bytecode into memory.
- Verifying bytecode for security.
- Converting bytecode into machine code using Just-In-Time (JIT) compiler.
- Executing the program.

Each operating system has its own JVM implementation, which allows the same bytecode to run on different platforms.

2) What is the byte code ? Discuss significance of byte code.

↳ Bytecode: It is an intermediate, platform-independent code generated by the Java compiler. It is stored in a (.class) file and is not understandable by the operating system directly.

→ Significance of Bytecode:

- Makes Java platform independent.
- Enables secure execution through bytecode verification.

Scanned with OKEN Scanner

- Improves portability of programs.
- Allows JVM to optimize performance using JIT compiler.
- Prevents direct access to hardware, improving safety.

Thus, bytecode plays a vital role in Java's security, portability and the performance.

3) List out features of Java. Explain any two features in details. Justify Java enables high performance.

↳ Java is a widely used, object-oriented programming language designed to be simple, secure, portable, and high performing. It provides many powerful features that make it suitable for developing robust and ~~sp~~ scalable applications.

→ Features of Java:

(i) Simple
(ii) Object Oriented
(iii) Platform Independent
(iv) Secure
(v) Robust
(vi) Portable
(vii) Multithreaded
(viii) Distributed
(ix) Dynamic
(x) High Performance.

Feature -1: Object Oriented ⇒ Java is purely object oriented (except primitive types). It uses concepts such as:

- Classes and Objects
- Encapsulation
- Inheritance

Scanned with OKEN Scanner

- Polymorphism
- Abstraction

These concepts help in code reusability, modularity, and easy maintenance.

Feature-2: Secure ⇒ Java provides a high level of security through:
- No use of pointers.
- Bytecode verification.
- Security Manager.
- Access control mechanisms.

→ Java enables high performance:

Java uses Just-In-Time (JIT) Compiler, which converts bytecode into native machine code at runtime. This increases execution speed. Efficient garbage collection and optimized memory management also contribute to Java's high performance.

4) What are the different types of errors in Java? Explain in details.

↳ Java errors are broadly classified into three types:
(i) Compile-Time Errors
(ii) Runtime Errors
(iii) Logical Errors.

(i) Compile-Time Errors: They are detected by the Java compiler during the compilation of the source code. These errors prevent the program from compiling successfully, so the program cannot be executed until they are corrected.

Causes of Compile-Time Errors:
- Syntax mistakes.
- Missing semicolons.

Scanned with OKEN Scanner

- Undeclared variables.
- Type mismatch errors.
- Incorrect use of keywords.

(ii) Runtime Errors: It occurs during the execution of the program. These errors are not detected by the compiler because the syntax is correct, but they occur due to invalid operations at runtime. Runtime errors are also known as exceptions in Java.

Common Runtime Errors:
- Arithmetic Exception
- Array Index Out of Bounds Exception
- Null Pointer Exception.
- Number Format Exception.

(iii) Logical Errors: It occur when the program executes successfully but produces incorrect or unexpected output due to faulty logic or incorrect algorithm.

Causes of Logical Errors:
- Wrong Formulas.
- Incorrect conditions.
- Improper loop logic.

5) What is Type Casting? Demonstrates with example.

↳ Type Casting in Java is the process of converting a value from one data type to another data type. It is commonly used when assigning a value of one type to a variable of another type.

Java supports typecasting to ensure type safety and data compatability during assignments and expressions.

Scanned with OKEN Scanner

→ Types of Type Casting in Java:
Java supports two types of type casting:
(i) Implicit Type Casting
(ii) Explicit Type Casting

(i) Implicit Type Casting (Widening Casting): Implicit type casting occurs automatically when a smaller data type is converted into a larger data type. There is no risk of data loss. This conversion is done by the Java compiler automatically.

Order of Widening:
byte → short → int → long → float → double

Example:
```
int a = 10;
double b = a;
System.out.println(b);
```

Here, the integer value 10 is automatically converted into a double value 10.0. No explicit instruction is required.

(ii) Explicit Type Casting (Narrowing Casting): Explicit type casting is required when a larger data type is converted into a smaller data type. There is a possibility of data loss. The programmer must specify the target data type manually.

Syntax: dataType variable = (dataType) value;

Example:
```
double x = 10.75;
int y = (int) x;
System.out.println(y);
```

Scanned with OKEN Scanner

The double value 10.75 is explicitly cast into an integer. The fractional part .75 is discarded, and the output becomes 10.

6) Explain Data Types in Details.

↳ In Java, data types specify the type of data a variable can store and the amount of memory allocated to it. Proper use of data types ensures efficient memory usage, data security, and error-free execution.

Java data types are broadly classified into two categories:
(i) Primitive Data Types
(ii) Non-Primitive Data Types

(i) Primitive Data Types: They store simple, single values and are predefined in Java. There are 8 primitive data types.

Data Type	Size	Description
byte	1 byte	Stores small integers (-128 to 127).
short	2 bytes	Stores larger integers than byte.
int	4 bytes	Stores whole numbers.
long	8 bytes	Stores very large integers.
Float	4 bytes	Stores decimal numbers (single precision).
double	8 bytes	Stores decimal numbers (double precision).
char	2 bytes	Stores a single character (Unicode).

Scanned with OKEN Scanner

boolean	1 bit	Stores true or false.

Example:

```
int age = 20;
float marks = 85.5f;
char grade = 'A';
boolean result = true;
```

(ii) Non-Primitive Data Types: They store references to objects rather than actual values. These are created by the programmer and are more flexible.

Common Non-Primitive Data Types

- String - Stores sequence of characters.
- Array - Stores multiple values of same type.
- Class - Blueprint of objects.
- Interface - Used for abstraction.

Example:

```
String name = "Java";
int [ ] arr = {1, 2, 3};
```

Characteristics:

- Can store multiple values.
- Have methods.
- Memory allocation is dynamic.
- Can be null.

7) Write a single program which shows the usage of following keywords:
(i) import (ii) new (iii) scanner (iv) break (v) continue

Scanned with OKEN Scanner

↳

```
import java.util.Scanner;
class KeywordDemo {
  public static void main(String[] args) {
    scanner sc = new Scanner(System.in);

    For (int i = 1; i <= 10; i++) {
      if (i == 4)
        continue;

      if (i == 8)
        break;

      System.out.println(i);
    }
  }
}
```

Output:

1
2
3
5
6
7

8) Differentiate between (i) while and do while (ii) break and continue
(iii) How many times is the println statement executed?

```
For (int i = 0; i < 10; i++)
For (int j = 0; j < i ; j++)
System.out.println (i * j);
```

Scanned with OKEN Scanner

(iv) If the value of variable x is 1 then what will be returned by the following expression:

↳ (i) Differentiate between while and do-while.

while	do-while
→ Entry-controlled loop.	→ Exit-controlled loop.
→ Condition checked first.	→ Condition checked last.
→ Executes zero or more times.	→ Executes at least once.
→ No semicolon after while.	→ Semicolon required after while.

→ (ii) Differentiate between break and continue.

break	continue
→ Exits the loop.	→ Skips current iteration.
→ Loop stops completely.	→ Loop continues.
→ Used to terminate loop.	→ Used to skip condition.

→ (iii) Outer loop runs from i = 0 to 9.

Inner loop runs i times for each value of i.

Scanned with OKEN Scanner

i	inner loop runs
0	0 times
1	1 time
2	2 times
3	3 times
4	4 times
5	5 times
6	6 times
7	7 times
8	8 times
9	9 times

Total executions:

$0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 = 45$

∴ println executes 45 times.

Scanned with OKEN Scanner